Lecture Slip Questions: Answers

CSCI 235 Software Design and Analysis II, Hunter College
Lecture slip of Tuesday, October 20, 2026 (State-Space Search)

Thank you for the questions on your slips. They are answered below, grouped by topic. The table of contents and the table on this page are links: click an entry to jump to its answer.

Your question	**Answer**
Can we use a cheat sheet during the exam?	Section 1
Could we have a summary of enqueue and dequeue?	Section 2
After we add `value` and `x`, do we multiply with `k+1`? How does the code do that?	Section 3
Can you go over questions 2 and 3?	Section 4, Section 5
Can you show a step-by-step or a diagram of `find()`?	Section 4
Can you give an overview of permutations, $\binom{n}{k}$, $n!$?	Section 6
Can you go over the cheat sheet and review today's lesson?	Section 7
Could we work on more tracing and search problems?	Section 8
More visual representations in the slides?	Section 9

Contents

1 Can we use a cheat sheet during the exam? **2**

2 Summary of enqueue and dequeue **2**

3 Slip question 1: how does `search` work? **2**

4 Slip question 2: `find` step by step **3**

5 Slip question 3: the `double` trap **4**

6 Permutations, $n!$, and $\binom{n}{k}$ **5**

7 Review of today's lesson and the cheat sheet **5**

8 More tracing practice **6**

9 Slides and feedback **6**

1 Can we use a cheat sheet during the exam?

Only the cheat sheet provided with the exam is allowed. You may not bring your own notes or a self-made cheat sheet. The provided sheet is printed on the last two pages of your exam packet (one sheet, front and back). Everything else is closed book and closed notes.

The version posted on the course schedule was **corrected on Oct 20**. Please re-download it, so that you study from the same version you will get on the exam.

2 Summary of enqueue and dequeue

The circular queue has capacity C and two indices: `front_` (the first item) and `back_` (the last item). An empty queue has `front_ == -1 && back_ == -1`.

Operation	If the queue is...	What happens
`enqueue`	full	nothing
`enqueue`	empty	`front_ = back_ = 0`
`enqueue`	otherwise	`back_ = (back_ + 1) % C`
`dequeue`	empty	nothing
`dequeue`	one item (`front_ == back_`)	`front_ = back_ = -1`
`dequeue`	otherwise	`front_ = (front_ + 1) % C`

Size. If `front_ <= back_`, the size is `back_ - front_ + 1`. If the queue has wrapped around (`front_ > back_`), the size is `C - front_ + back_ + 1`. The queue is full when the size equals C.

Example ($C = 4$, starting empty): `enqueue(5)` gives $(0, 0)$; `enqueue(7)` gives $(0, 1)$; `dequeue()` gives $(1, 1)$; `enqueue(1)` gives $(1, 2)$; `enqueue(2)` gives $(1, 3)$; `enqueue(9)` gives $(1, 0)$, because `back_` wrapped around to index 0.

This is the Stacks and Queues section of the cheat sheet.

3 Slip question 1: how does `search` work?

The question: "After we add `value` and `x`, do we multiply with `k+1`? How does the code do that?"

Short answer: nothing is multiplied by `k+1`. Two different things travel through the calls:

- `k` is a **position**: which number of `v` comes next. `k + 1` means "move on to the next number".
- `value` is the **running result** so far. `value + x` is the new running result after adding the next number.

The four calls joined by `||` are four **separate branches**: add, subtract, multiply, divide. Multiplication is one of the four choices; it does not come after addition.

```
return search(v, k + 1, value + x, target) // branch 1: add
    || search(v, k + 1, value - x, target) // branch 2: subtract
    || search(v, k + 1, value * x, target) // branch 3: multiply
    || (x != 0 && search(v, k + 1, value / x, target)); // branch 4: divide
```

Trace for `v = {3, 4, 2}`, target 14, first call `search(v, 1, 3, 14)`:

Step	Call	Next number `x`	What happens
1	`search(v, 1, 3, 14)`	$v[1] = 4$	try add first: $3 + 4 = 7$
2	`search(v, 2, 7, 14)`	$v[2] = 2$	try add first: $7 + 2 = 9$
3	`search(v, 3, 9, 14)`	—	$k = 3$ is a leaf; $9 \neq 14$, return false
4	back in step 2	2	try subtract: $7 - 2 = 5$, leaf, $5 \neq 14$, false
5	back in step 2	2	try multiply: $7 \times 2 = 14$, leaf, $= 14$, **true**
6	`||` stops at the first true: no other branch is tried.		

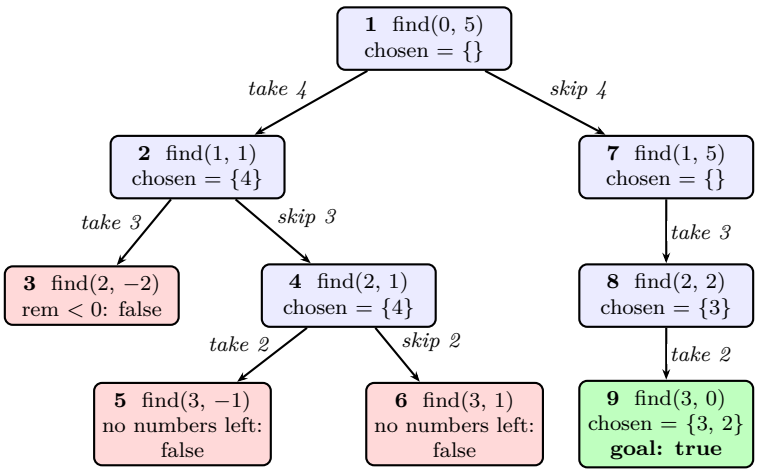
So the search checks 3 leaves (9, 5, 14) out of $4 \times 4 = 16$ possible.

4 Slip question 2: `find` step by step

The questions: "Can you go over question 2?" and "Can you show a step-by-step or a diagram of `find()`?"

```
bool find(const std::vector<int>& v, int k, int rem, std::vector<int>& chosen) {
  if (rem == 0) { return true; }                              // goal
  if (k == static_cast<int>(v.size()) || rem < 0) { return false; } // dead end
  chosen.push_back(v[k]);                                     // choose
  if (find(v, k + 1, rem - v[k], chosen)) { return true; }    // explore: take v[k]
  chosen.pop_back();                                          // undo
  return find(v, k + 1, rem, chosen);                         // explore: skip v[k]
}
// v = {4, 3, 2}; first call: find(v, 0, 5, chosen)
```

The idea. At each number the function has two choices: **take** it (subtract it from `rem`) or **skip** it. It tries "take" first. If that branch fails, it **undoes** the choice and tries "skip". Each call is written as `find(k, rem)`. Red nodes are dead ends. The green node is the goal. The small numbers show the order of the calls.



Step by step, with the undo included:

Call	Action	`chosen` after	Result
1 `(0,5)`	take 4	{4}	explore call 2
2 `(1,1)`	take 3	{4, 3}	explore call 3
3 `(2,−2)`	`rem < 0`: dead end	{4, 3}	false; **undo 3**: {4}
4 `(2,1)`	take 2	{4, 2}	explore call 5
5 `(3,−1)`	no numbers left	{4, 2}	false; **undo 2**: {4}
6 `(3,1)`	no numbers left	{4}	false, so call 4 and call 2 fail
1	**undo 4**, then skip 4	{}	explore call 7
7 `(1,5)`	take 3	{3}	explore call 8
8 `(2,2)`	take 2	{3, 2}	explore call 9
9 `(3,0)`	`rem == 0`: goal	{3, 2}	**true**, returned up the stack

Answers to the slip.

(a) `chosen = {3, 2}`, and $3 + 2 = 5$.

(b) Without `chosen.pop_back();`, the dead ends are never erased. The final `chosen` is `{4, 3, 2, 3, 2}`, whose sum is 14, not 5. The function still returns `true`, but the recorded answer is wrong, because every later branch starts from a state that is not the one where we branched.

The rule to remember: every `push_back` (choose) needs a matching `pop_back` (undo) on the failure path.

5 Slip question 3: the `double` trap

```
double a = 0.5 + 0.25;
std::cout << (a == 0.75);              // line 1
double s = 0.0;
for (int i = 0; i < 10; i++) { s += 0.1; }
std::cout << (s == 1.0);               // line 2
```

(a) Line 1 prints `1`. Line 2 prints `0`.

(b) Computers store numbers in binary. $0.5 = 2^{-1}$ and $0.25 = 2^{-2}$, and their sum 0.75 is also a sum of powers of two, so all three are stored **exactly**. But 0.1 is not a finite binary fraction, just as 1/3 is not a finite decimal (you write 0.333 and lose a little). Each `s += 0.1` adds a tiny rounding error, and after ten additions `s` is about 0.99999999999999989, not exactly 1.0.

(c) Compare with a small tolerance: `std::abs(s - 1.0) < 1e-9`. This is the same test the search uses for its goal: `std::abs(value - target) < 1e-9`.

6 Permutations, $n!$, and $\binom{n}{k}$

(Permutations are the topic of Class 15 on Thursday, Oct 22. They are not on the midterm.)

Use 4 guests, A, B, C, D.

Formula	Meaning	Example with 4 guests
$n! = n \times (n-1) \times \cdots \times 1$	arrangements of n *different* items in a row (order matters)	4 guests in 4 chairs: $4 \times 3 \times 2 \times 1 = 24$
$\dfrac{n!}{(n-k)!}$	arrangements of k items chosen from n (order matters)	2 of the 4 guests in 2 chairs: $4 \times 3 = 12$
$\dbinom{n}{k} = \dfrac{n!}{k!\,(n-k)!}$	ways to *choose* k items from n (order does *not* matter)	choose 2 of the 4 guests: $12 \div 2 = 6$

How they connect. The 12 ordered pairs include each unordered pair twice (AB and BA are the same choice), so $\binom{4}{2} = 12/2! = 6$. In general, $\binom{n}{k}$ is the number of ordered arrangements divided by $k!$, the number of ways to reorder the chosen items.

Repeated items. If some items are identical, divide by the factorial of each repeat count. For `"AAB"`: $3!/2! = 3$ distinct arrangements (`AAB`, `ABA`, `BAA`).

7 Review of today's lesson and the cheat sheet

Today's lesson (Oct 20), in three ideas.

1. **A search problem has states, moves, and a goal.** In the maze, a state is the cell you stand on; in Make 24, it is the numbers left on the table.
2. **Choose, explore, undo.** A recursive search makes a choice, explores what follows, and undoes the choice so that the next branch starts from the same state (Section 4).
3. **Compare floating-point numbers with a tolerance,** never with `==` (Section 5).

State-space search is **not** on the midterm (Oct 27), so it is not on the cheat sheet.

What is on the cheat sheet. It has seven sections:

1. Classes and polymorphism (`virtual`, `override`, slicing, the copy constructor and copy assignment)
2. Templates, MiniVector, and memory (Rule of Three, growth by doubling)
3. Singly linked list (traversal, reversal, `popFront`)
4. Stacks and queues (the circular queue rules of Section 2)
5. Recursion (base case, recursive step, call stack)
6. P2: DFS and BFS on a maze
7. Big-O patterns

How to review with it. Redo the Oct 15 and Oct 20 slip questions with the cheat sheet open, and notice which line of the sheet you reach for. The sheet reminds you of rules; you still need to practice applying them.

8 More tracing practice

Use the same function `find` from Section 4 with `v = {2, 3, 4}` and the first call `find(v, 0, 6, chosen)`.

(a) Draw the call tree like the one in Section 4. What is `chosen` when the call returns `true`?

(b) Delete `chosen.pop_back();`. What is `chosen` now, and is it a correct answer?

You can also redo question 2 of the Oct 15 slip (the function `g(n)`) and draw its call tree.

Check your work. (a) `chosen = {2, 4}`, because $2 + 4 = 6$. The search takes 2, takes 3, takes 4 (dead end, undo), skips 4 (dead end), undoes 3, skips 3, takes 4, and reaches `rem == 0`. (b) Without the undo, `chosen = {2, 3, 4, 4}`, whose sum is 13, not 6. The function still returns `true`.

9 Slides and feedback

Thank you for the feedback. More diagrams will be added to the slides, and the Class 15 slides already have trees for the arrangements.